Duelo de Contextos: Processos vs. Threads

Fabrine Drescher
Curso de Sistemas Operacionais

Abstract

This report presents an exploratory experiment comparing POSIX threads and POSIX processes in a Unix-like environment. The study evaluates creation overhead, communication and synchronization cost, and data consistency using a shared counter incremented up to one billion operations. Four scenarios were implemented: threads without synchronization, threads with `pthread_mutex`, processes without synchronization, and processes with a named POSIX semaphore. The results show that unsynchronized execution is faster but incorrect, while synchronization guarantees correctness at the cost of significant overhead, especially for processes using semaphores.

Resumo

Este relatório apresenta um experimento exploratório comparando threads POSIX e processos POSIX em ambiente Unix-like. A avaliação considera overhead de criação, custo de comunicação e sincronização, e consistência de dados usando um contador compartilhado incrementado até um bilhão de operações. Foram implementados quatro cenários: threads sem sincronização, threads com `pthread_mutex`, processos sem sincronização e processos com semáforo POSIX nomeado. Os resultados mostram que a execução sem sincronização é mais rápida, porém incorreta, enquanto a sincronização garante corretude ao custo de maior overhead, especialmente no caso de processos com semáforo.

1 Introdução

Processos e threads são mecanismos fundamentais de concorrência em sistemas operacionais. Embora ambos permitam a execução concorrente de tarefas, eles possuem diferenças importantes em termos de isolamento, compartilhamento de memória, custo de criação e mecanismos de comunicação.

O objetivo deste trabalho é comparar, de forma experimental, o comportamento de processos e threads no problema clássico do contador compartilhado. Nesse problema, várias unidades de execução incrementam um mesmo contador até o valor final esperado de 1.000.000.000. O experimento foi executado com $N = 2$, $N = 4$ e $N = 8$ unidades de execução.

A análise considera não apenas o tempo de execução, mas também a corretude do resultado final. Dessa forma, é possível observar a diferença entre uma execução rápida, porém inconsistente, e uma execução sincronizada, porém mais custosa.

2 Metodologia

Foram implementadas duas versões principais do experimento: uma baseada em threads POSIX, usando `pthread`, e outra baseada em processos POSIX, usando `fork`. Em ambas, o trabalho total foi dividido entre N unidades de execução.

Os quatro cenários avaliados foram:

- **T1**: threads sem sincronização;
- **T2**: threads com sincronização usando `pthread_mutex`;
- **P1**: processos sem sincronização, usando memória compartilhada;
- **P2**: processos com sincronização usando semáforo POSIX nomeado.

Em T1 e P1, o objetivo é observar a ocorrência de condições de corrida. Em T2 e P2, o objetivo é verificar o custo de garantir exclusão mútua sobre a seção crítica responsável por incrementar o contador.

3 Implementação

3.1 Threads

Na implementação com threads, o contador é uma variável global compartilhada naturalmente entre todas as threads do processo. No cenário T1, as threads incrementam o contador sem nenhum mecanismo de proteção. No cenário T2, o incremento é protegido por um `pthread_mutex`.

Listing 1: Seção crítica protegida por mutex em T2.

```
pthread_mutex_lock(&mutex);
counter++;
pthread_mutex_unlock(&mutex);
```

O uso do mutex garante que apenas uma thread por vez atualize o contador. Isso corrige a inconsistência, mas reduz o paralelismo efetivo, pois todos os workers disputam a mesma seção crítica.

3.2 Processos

Na implementação com processos, o contador precisa ser compartilhado explicitamente entre os processos filhos. Para isso, foi utilizada memória compartilhada System V, com as chamadas `shmget` e `shmat`.

Listing 2: Exemplo simplificado de uso de memória compartilhada.

```
int shmid = shmget(IPC_PRIVATE, sizeof(long long), IPC_CREAT | 0600);
void *ptr = shmat(shmid, NULL, 0);
```

No cenário P1, os processos atualizam o contador compartilhado sem sincronização. No cenário P2, a seção crítica é protegida por um semáforo POSIX nomeado criado com `sem_open`.

Listing 3: Seção crítica protegida por semáforo em P2.

```
sem_wait(sem);
(*counter)++;
sem_post(sem);
```

4 Ambiente de Execução

Os experimentos foram compilados e executados em um ambiente Unix-like com as características apresentadas na Tabela 1.

Tabela 1: Ambiente de execução utilizado nos testes.

Item	**Valor**
Sistema Operacional	macOS 15.6.1 24G90
Kernel	Darwin 24.6.0
CPU	Apple M1
Núcleos físicos/lógicos	8/8
Memória RAM	8 GB
Compilador	Apple clang 17.0.0

A assinatura do hardware foi registrada com o comando `sysctl -a | grep hw.ncpu`, obtendo a saída `hw.ncpu: 8`.

5 Resultados

A Tabela 2 apresenta os tempos de execução e o valor final do contador para cada cenário. O valor esperado em todos os casos era 1.000.000.000.

Tabela 2: Resultados obtidos nos experimentos.

Cenário	**N**	**Tempo (ms)**	**Contador Final**	**Correto?**
T1 - Threads sem sync	2	768,458	500.781.324	Não
T1 - Threads sem sync	4	407,375	251.557.444	Não
T1 - Threads sem sync	8	425,325	208.786.584	Não
T2 - Threads com mutex	2	14.983,797	1.000.000.000	Sim
T2 - Threads com mutex	4	28.011,320	1.000.000.000	Sim
T2 - Threads com mutex	8	25.543,228	1.000.000.000	Sim
P1 - Processos sem sync	2	685,323	500.691.727	Não
P1 - Processos sem sync	4	375,707	259.977.193	Não
P1 - Processos sem sync	8	379,196	197.772.466	Não
P2 - Processos com semáforo	2	1.573.728,967	1.000.000.000	Sim
P2 - Processos com semáforo	4	1.683.384,402	1.000.000.000	Sim
P2 - Processos com semáforo	8	2.596.231,994	1.000.000.000	Sim

A Figura 1 apresenta o gráfico de escalabilidade gerado a partir dos resultados experimentais.

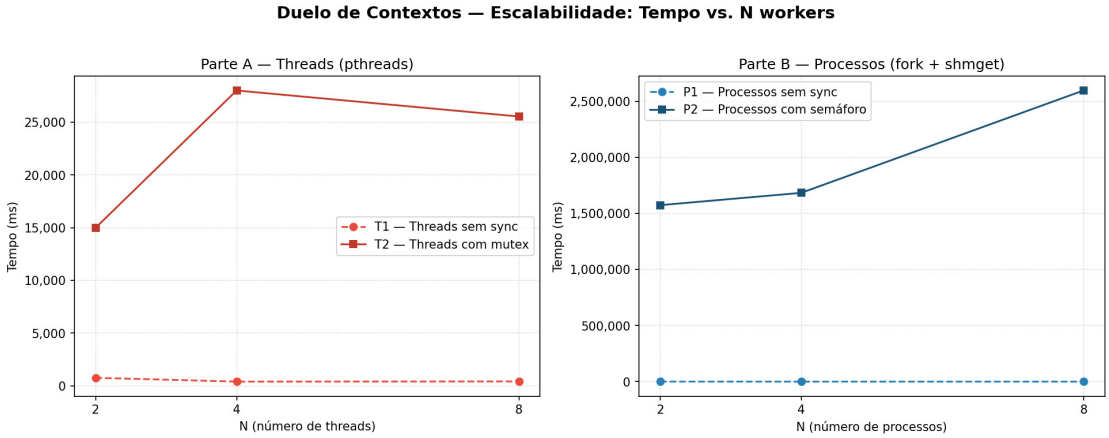

Figura 1: Tempo de execução em função do número de unidades de execução.

6 Análise

6.1 Consistência de Dados

Os cenários T1 e P1 produziram valores finais incorretos. Isso ocorre porque a operação `counter++` não é atômica. De forma simplificada, ela envolve três etapas: ler o valor atual da memória, somar uma unidade e escrever o novo valor de volta na memória.

Quando duas ou mais unidades de execução realizam essas etapas simultaneamente, pode ocorrer uma condição de corrida. Por exemplo, dois workers podem ler o valor 100, calcular 101 e gravar 101. Nesse caso, dois incrementos foram executados, mas o contador aumentou apenas uma unidade. O resultado é a perda de incrementos.

Em T1, o contador final caiu de 500.781.324 com duas threads para 208.786.584 com oito threads. Em P1, o comportamento foi semelhante: o contador caiu de 500.691.727 com dois processos para 197.772.466 com oito processos. Isso mostra que o aumento do número de workers aumenta a disputa pelo contador compartilhado e torna a inconsistência mais evidente.

6.2 Custo de Sincronização

Nos cenários T2 e P2, o valor final foi sempre correto. Entretanto, os tempos de execução aumentaram significativamente, pois cada incremento do contador passou a exigir sincronização.

Em T2, o uso de `pthread_mutex` garante exclusão mútua entre threads. Como todas as threads disputam a mesma seção crítica, apenas uma pode incrementar o contador por vez. Isso torna o programa correto, mas reduz o paralelismo efetivo.

Em P2, o custo foi ainda maior. O semáforo POSIX nomeado protegeu corretamente a seção crítica entre processos, mas cada incremento exigiu operações `sem_wait` e `sem_post`. Como o experimento executa um bilhão de incrementos, o overhead acumulado tornou-se muito elevado.

6.3 Overhead de Criação e Comunicação

Threads geralmente possuem menor custo de criação e comunicação mais simples, pois compartilham o mesmo espaço de endereçamento do processo principal. Processos, por outro

lado, oferecem maior isolamento, mas exigem mecanismos explícitos de comunicação entre processos, como memória compartilhada, pipes, sockets ou semáforos.

Nos resultados sem sincronização, T1 e P1 apresentaram tempos próximos. Isso não significa que processos sejam necessariamente mais baratos que threads. Neste experimento, o tempo total é dominado principalmente pelo grande número de incrementos, pela contenção na memória e pelo comportamento do escalonador do sistema operacional. Portanto, o experimento mostra o comportamento geral dos cenários, mas não isola apenas o custo puro de criação.

6.4 Escalabilidade

Nos cenários sem sincronização, o aumento de N reduziu o tempo de execução de $N = 2$ para $N = 4$, mas os valores finais permaneceram incorretos. Assim, houve ganho de tempo, porém sem garantia de consistência.

Nos cenários com sincronização, o aumento de N não gerou melhora proporcional. Isso ocorre porque o contador é um único recurso compartilhado. Mesmo com várias unidades de execução, apenas uma pode modificar o contador por vez quando a sincronização está ativa. Assim, o mecanismo de sincronização se torna o principal gargalo do experimento.

7 Conclusão

Os experimentos demonstram que desempenho e corretude precisam ser avaliados em conjunto. Os cenários T1 e P1 foram mais rápidos, mas produziram resultados incorretos devido a condições de corrida. Já os cenários T2 e P2 produziram corretamente o valor final de 1.000.000.000, mas apresentaram tempos de execução muito maiores.

A principal conclusão é que a ausência de sincronização pode tornar o programa mais rápido, mas compromete a consistência dos dados. Por outro lado, a sincronização garante corretude, mas adiciona overhead e pode reduzir significativamente a escalabilidade.

Também foi possível observar que threads favorecem comunicação direta e menor overhead, enquanto processos oferecem maior isolamento ao custo de mecanismos explícitos de comunicação e sincronização. No experimento realizado, o uso de semáforo POSIX nomeado em processos apresentou o maior custo, principalmente porque a sincronização foi realizada a cada incremento do contador.

Uma implementação otimizada poderia reduzir o overhead acumulando incrementos em contadores locais e sincronizando apenas ao final. Entretanto, essa estratégia mudaria o foco do experimento, que era justamente observar os efeitos da concorrência e da sincronização sobre um contador compartilhado.

Referências

[1] TANENBAUM, A. S.; BOS, H. *Modern Operating Systems*. 4. ed. Pearson, 2015.

[2] SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Operating System Concepts*. 10. ed. Wiley, 2018.

[3] IEEE and The Open Group. *POSIX Threads*. The Open Group Base Specifications.

[4] IEEE and The Open Group. *POSIX Semaphores*. The Open Group Base Specifications.